



2

9

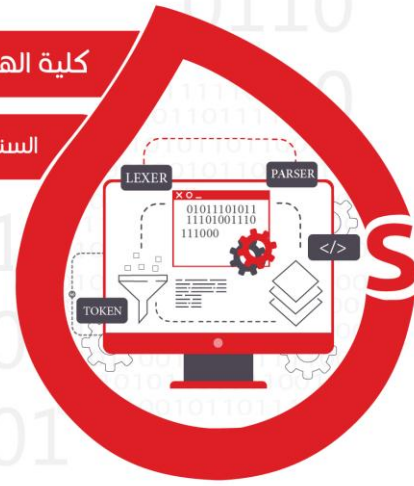
1080 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Symbol table



م.وسيم البرزة، آية شحادة

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 27/5/2025

الترجمات

Symbol Table

بنية بيانات تستخدم لأغراض متعددة في البرمجة، من أبرزها تتبع المتغيرات والدوال أثناء تحليل الشيفرة. فعندما يكون لدينا شجرة بناء نحوي مجردة (AST) كبيرة، تصبح عملية البحث عن الرموز مكلفة من حيث الوقت والذاكرة. لذلك، يتم استخدام الـ Symbol Table لتخزين المعلومات المهمة التي نحتاج التحقق منها بشكل متكرر، وهو ما يعرف بعملية **التتبع (Tracking)**. كما تستخدم أيضاً في **تخزين المعلومات (Storing Info)** المتعلقة بالرموز، مما يسمح بالوصول إليها لاحقاً بكفاءة أعلى وبتكلفة أقل.

الفائدة الأساسية من الـ symbol table

1. Scope management

بهدف تنظيم الرموز ومنع حدوث تعارض أو أخطاء في التعريف. فعلى سبيل المثال: إذا كان لدينا متغير x مكرر في أكثر من موقع داخل نفس النطاق: نضع scope لنحدد أي x هي التي نحتاج طباعتها نهائياً

`int x = 0;`

`int x = 10;`

في هذا المثال، كلا التعريفين للمتغير x يقعان ضمن نفس الـ scope، مما يؤدي إلى تعارض يمنع المترجم من تحديد أي من المتغيرين يجب استخدامه.

الـ Symbol Table تساعد في تنظيم هذه الرموز من خلال ربط كل متغير بنطاقه الصحيح، وبالتالي تحديد المتغير المقصود بدقة عند الاستخدام.

2. Error checking

في المثال السابق، الكود يعد صحيح من حيث البنية النحوية (Syntax)، لكنها تحتوي على خطأ دلالي (Semantic Error)، مثل تعريف نفس المتغير x مرتين داخل نفس النطاق. بدون استخدام الـ Symbol Table، قد يصعب على المترجم تحديد طبيعة الخطأ أو موقعه بدقة.

أما باستخدام الـ Symbol Table، يتم تصنيف هذا النوع من الأخطاء كـ "تعريف مكرر في نفس النطاق"، مما يمكن المترجم من اكتشاف الخطأ بسرعة وتحديد سببه بدقة، وبالتالي تسهيل معالجته



وأكثر استخدام شائع لـ Symbol Table هو التأكد من الاستخدام الصحيح لـ Identifiers (مثل أسماء المتغيرات والدوال). فهي تتيح للمترجم التحقق مما إذا تم تعريف المعرف قبل استخدامه، وإذا كان استخدامه متوافقاً مع نوعه ونطاقه، مما يساعد على منع الأخطاء الدلالية وضمان صحة البرنامج.

العمليات على الـ symbol table

operations	List, Map,.....
allocate	إنشاء Symbol Table جديد Allocate new empty symbol table
free	تستخدم لحذف شيء ما Remove all entries
Look up	تستخدم للبحث ضمن list او map ومقارنتها بحال وجود اسمين متشابهين Search for name entry
insert	تستخدم لوضع قيمة أو خلق قيمة ضمن الـ symbol table Insert a name
Set_attribute	تستخدم لإعطاء قيمة أو سمة إضافية لمدخل محدد، مثل نوع المتغير أو نطاقه. Set attribute to give entries
get_attribute	تستخدم لاسترجاع معلومات أو صفات مرتبطة بمعرف معين، مثل نوعه أو مكان تعريفه. get attribute for give entries

مثال على بنية الـ symbol table

`int x = 0;`

variable	type	value
<code>x</code>	<code>int</code>	

لنفرض تم الحفظ ثم قمنا
بكتابة: `int x = 10;`

variable	type	value
<code>x</code>	<code>int</code>	<code>0</code>

هنا القيمة صفر لكن اضعنا 10 فيصبح لدينا semantic error
بحاجة للتصحيح عن طريق الـ scope



implementation

لدينا الكثير من البنس التي يمكن استخدامها ضمن التخزين منها:

1. List: تعد مناسبة لتخزين البيانات فقط، لكنها غير فعالة في عمليات البحث، لأن كلفة البحث فيها هي $O(n)$ ، مما يجعلها غير ملائمة للتطبيقات التي تتطلب بحثاً متكرراً.

2. Map: مثالية في عمليات البحث، حيث تكون كلفة البحث فيه $O(1)$ في المتوسط. ويمكن استخدام Hash Map كإصدار متطور منها لزيادة الكفاءة والمرونة.
3. LinkedList
4. Binary search tree
5. وغيرها الكثير

مثال:

```
class VariableSymbolTable {
    HashMap < String,String > table;
```

// allocate - Constructor: إنشاء جدول رموز جديد باستخدام Map

```
public VariableSymbolTable() {
    table = new HashMap <> ();
}
```

// free - removeAll: حذف جميع العناصر من جدول الرموز

```
public void removeAll() {
    table.clear();
}}
```

الشرح:

نستخدم بنية HashMap لأنها مناسبة لعمليات البحث السريعة.

لإنشاء جدول الرموز، نستخدم الـ Constructor لتخصيص وتخصيص الذاكرة، وهو ما يُعرف بعملية allocate. وعند الرغبة في حذف جميع العناصر داخل جدول الرموز، نستخدم الدالة removeAll() والتي تقوم باستدعاء clear() من الـ Map، وهذا يُمثل عملية free في الـ Symbol Table.remove.

- عند استخدام دالة Insert(key, value) لإضافة عنصر جديد إلى الـ Symbol Table، يجب أولاً التحقق مما إذا كان المفتاح (key) موجوداً مسبقاً. وذلك لتفادي حدوث double allocation، أي محاولة تعريف نفس المعرف مرتين داخل نفس النطاق، مما قد يؤدي إلى أخطاء في التنفيذ أو الترجمة.



You only live once, but if you do it right, once is enough.

مثال على إدارة ال Scope باستخدام Symbol Table :

```

Scope {
Stack < Map < String, String >> {
    x { → push
        x
    } → pop
        x
    }
}

```



العمليات الأساسية في إدارة ال: Scope

1. Enter Scope
2. إضافة متغير
3. Exit Scope
4. Update (Insert)

أهمية إنشاء ال Symbol Table خارج ال Visitor لضمان التناسق والكفاءة

عند العمل مع ال-Visitor Pattern، من المهم جداً إنشاء كائن ال Symbol Table خارج الزائر (Visitor) وليس داخله. السبب هو أنه إذا تم إنشاء ال Symbol Table داخل الزائر، فسيتم إنشاء نسخة جديدة منه مع كل زيارة (visit)، مما يؤدي إلى وجود عدة جداول منفصلة بدلاً من جدول واحد يحتوي على جميع المعلومات المرتبطة بالتحليل

```

visitBlock(ctx) {
    enterScope();

    // تنفيذ العمليات داخل هذا النطاق

    B = new Block(...);
    exitScope();
    return B;
}

```



ملاحظة:

ال (AST (Abstract Syntax Tree) تختلف تماماً عن ال Symbol Table من حيث الوظيفة، فالجداول لا تستخدم كبديل للشجرة، بل كأداة مساعدة لتسهيل الوصول إلى المعلومات أثناء عملية التحليل. فبدلاً من المرور على كامل الشجرة - خصوصاً عندما تكون ضخمة - يمكنك استخدام ال Symbol Table للذهاب مباشرة إلى المعرف أو المعلومة المطلوبة دون الحاجة لتكرار الزيارة الكاملة للـ AST.

Semantic Error

هي الأخطاء التي تظهر بالrun time أي أخطاء غير منطقية تظهر بالسياق

مثال:

```
int x = 0;
int x = 10;
```

كيف نعالج خطأ semantic:

أولاً check ثم اتحقق ثم اخزن

1. Create symbol table

لكل خطأ يوجد لدينا جدول

```
Class SymbolTable{
    HashMap < String,String > table;
    public VariableSymbolTable() {
        table = new HashMap <> ();
    }
    public void removeAll() {
        table.clear();
    }
}
```

2. visitor

```
visitDeclaration(ctx) {
    String x = ctx.ID().getText();

    // التحقق مما إذا كان x معرفاً مسبقاً ضمن الـ Symbol Table
    if (SymbolTable.check(x)){
        // في حال كانت القيمة true، فهذا يعني أن المعرف موجود مسبقاً → خطأ
        ErrorHandler.logError(new TypeException("Variable '" + x + "' already declared."));
    } else {
        // إذا كانت false، فالأمر سليم ويمكن إضافة x
        SymbolTable.insert(x);
    }
}
```

3. checking



4. error is exist

في حال اكتشاف وجود خطأ أثناء التحليل، يتم إنشاء ملف أو سجل يحتوي على تفاصيل الخطأ، بما في ذلك رقم السطر الذي حدث فيه.

يمكن الحصول على رقم السطر باستخدام الدالة:

```
ctx.ID().getSymbol().getLine();
```

أما لتحديد العمود (أي موقع الخطأ ضمن السطر)، نستخدم:

```
ctx.ID().getSymbol().getColumn();
```

ملاحظة: تطبيقك يجب أن يحوي على الأقل 5 أخطاء ومعالجتها بخمس جداول

ملاحظات:

- لتجنب التعقيدات الناتجة عن وجود أخطاء أثناء التحليل، يفضل اتباع الآلية التالية:
- 1. تنظيم الملفات:
 - أنشئ 3 ملفات:
 - الملف الأصلي: يحتوي على الكود الأساسي الصحيح دون أي أخطاء.
 - ملف الأخطاء: (Errors File) يُسجل فيه تفاصيل جميع الأخطاء التي تم اكتشافها أثناء التحليل.
 - ملف النتائج النهائية: (EndResult) يستخدم لتخزين المخرجات الصحيحة من التحليل في حال لم تكتشف أخطاء.
- 2. التعامل مع الأخطاء:
 - عند اكتشاف خطأ، يتم إنشاء كلاس مخصص للخطأ. (Exception class)
 - داخل هذا الكلاس، نستخدم `FileWriter` (للكتابة) و `FileReader` (للقراءة) حسب الحاجة.
 - يتم تعريف تابع `logError(Exception e)`، حيث يقوم بـ:
 - استخدام `e.getMessage()` للحصول على نص الخطأ.
 - تنفيذ عملية `append` لهذا النص إلى ملف الأخطاء باستخدام `FileWriter`.
- 3. استخدام `ErrorHandler`:
 - عند ظهور الخطأ، نستدعي:


```
ErrorHandler.logError(new RareTypeException(...));
```
 - بحيث يكون `RareTypeException` كلاس يُمدد `Exception` (أي `extends Exception`)



KEEP_GOING

تصحيح طريقة تعريف الـ label داخل القواعد النحوية: (Grammar)

- الطريقة **الخاطئة** في التعبير عن الـ label تكون على الشكل التالي:

```
ImportState: (.....|.....|.....);
```

في هذا الشكل، لا يتمكن الـ parser من تعيين label واضح ومحدد لكل خيار من خيارات القاعدة، مما يؤدي إلى صعوبة في التمييز بينها أثناء التحليل أو الزيارة.

- الصيغة **الصحيحة** لتعريف الـ label هي بوضع كل فرع من القاعدة على سطر منفصل، مع تسمية (label) مميزة له، كما يلي:

```
ImportState:
```

```
.....
|.....
|.....
;
```

- يتم إعطاء label خاص بكل خيار.
- يُسهل على الـ visitor التعرف على نوع الفرع المستخدم عند تنفيذ الزيارة
- يمنع حدوث التباس أو أخطاء تحليل لاحقة

مثال عن label :

```
ImportState:
```

```
exp '/' exp    #Arch
|exp '*' exp    #Arch
|exp '+' exp    #Arch
|ld             #ld
;
```

في هذه الحالة لدينا visitor لكل قاعدة أي لدينا 4 visitors حسب عدد القواعد

- في حال وضع mal في قاعدة الضرب كالتالي

```
exp '*' exp    #mal
```

هل يمكن أن اعطي ذات الاسم للقاعدة التي تليها '+'؟؟

```
exp '+' exp    #mal
```

أجل يمكن ذلك وبهذه الحالة القاعدتين معاً لهما visitor واحد مما يعني يصبح لدينا 3 visitor بدلاً من 4 visitor

Almost done



ملاحظة: في ANTLR ، يمكن تعريف قاعدة واحدة تحتوي على عدة فروع، ولكل فرع نُعطي label مميز. بهذه الطريقة يتم إنشاء Visitor method لكل label على حدة. كما يمكن كتابة عدة قواعد تحمل نفس الاسم (خيارات داخل نفس القاعدة)، مما يُنتج زائر واحد إذا استخدم label موحد. هذا يُسهّل تنظيم الزوّار ويقلل التكرار في ملف الـ Visitor

مثال على AST:

```
Interface public Exception{ }
Class public Arch{
    String op;
    exp expLeft;
    exp expRight;
}
Class Identifier {
    String value;
}
```

نوع العملية المراد تخزينها {



الشرح:

لماذا تم وضع Interface بدلاً من class؟؟
تم استخدام interface بدلاً من class لأن جميع القواعد تعتبر تعابير (expressions)، لذلك من الأفضل تعريف واجهة مشتركة تجمعها بدل إنشاء كلاس عام يحتوي على متغير id ومصفوفة exp لكل حالة، وهو ما يؤدي إلى تعقيد في التخزين والتنفيذ.
ولهذا السبب، تم تعريف ثلاث كلاسّات منفصلة (Arch, Identifier, ...)، كل منها يمثل label مختلف حسب الاستخدام. وعلى الرغم من أنه يمكن دمج جميع القواعد داخل كلاس واحد، إلا أن ذلك سيجعل عملية التخزين والاستخدام أكثر تعقيداً.

مثال على visitor:

```
VisitArch(ctx) {
    Arch arch = new Arch();
    arch.leftExp = visit(ctx.exp(0));
    arch.op = ctx.getChild(1).getText();
    arch.rightExp = visit(ctx.exp(1));
    return arch;
}

VisitID(ctx) {
    String value = ctx.getText();
    return new ID(value);
}
```

// الطرف الأيسر
// العملية (مثل + أو -)
// الطرف الأيمن
// ID إرجاع قيمة الـ

الشرح:

- في هذا المثال، لدينا زائران: (visitors)
- الأول VisitArch يعالج العمليات الثنائية (مثل الجمع أو الطرح) ويحدد طرفي العملية (leftExp و rightExp) ونوع العملية نفسها. (op)
- الثاني VisitID يتعامل مع المعرفات (ID) ، ويقوم بإرجاع القيمة النصية للمعرف الموجود داخل التعبير.

سؤال:

إذا كانت القاعدة تحتوي على عدد كبير من التعبيرات (expressions) ، كيف يمكن جلب تعبير معيّن منها؟

الجواب:

عند وجود أكثر من تعبير ضمن قاعدة واحدة، يتم تمثيلهم ك قائمة من التعبيرات (List of expressions) داخل ctx وفي هذه الحالة، لا نقوم بزيارة كل تعبير يدويًا، بل نستخدم حلقة للوصول إليهم جميعًا:

```
List < Exp > expressions = new ArrayList <> ();
for (int i = 0; i < ctx.exp().size(); i++) {
    expressions.add(visit(ctx.exp(i)));
}
```

نستخدم هذا الأسلوب فقط عندما يكون هناك عدد غير محدد أو كبير من التعبيرات داخل القاعدة. أما في الحالات البسيطة (مثل مثالنا السابق الذي يحتوي فقط على leftExp و rightExp)، فنصل لكل تعبير بشكل مباشر دون الحاجة لحلقة.

THE_END_SEE_U_SOON

